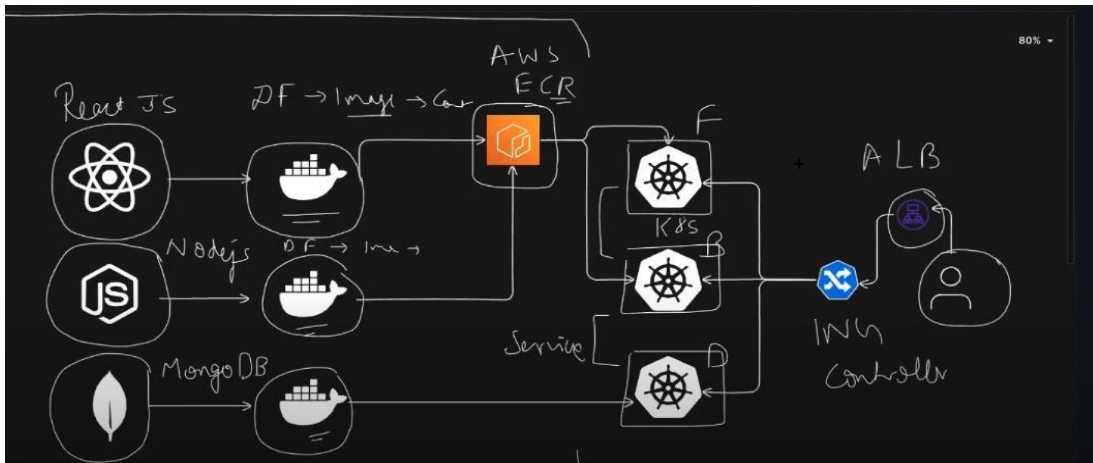


DevOps Project LIVE – 3-Tier Application : Deployment on Kubernetes

Tws.



Tiers means = 1. **web tier** (presentation) Generally Front end rkhte hai
2. **Logical Tier** (API or Business logic)
3. **Database tier**.

For example:

- **Frontend** = React js
- **backend** = Node js
- **Database** = Mongo DB ,SQL Depends on Your application .

Docker files → Image → Container

@ Lets create **Dockerfile** each tier **Except mongo db** because **image already available**.

- **React js** ki Docker file likhege.
- **Node js** ki Docker file likhege
- **MongoDb** ki docker file **nhi likhege** mongo db ki **image already available** hai **dockerhub** mai

ECR : mai **store** ho rhi hai **docker image** mai (Public, Private)

Deploy in Kubernetes = KinD, minikube, kubeadm are **Good for local deployment** but, to create cluster many ways we can create it .

Inside Cluster we have **front end, backend and DB** these **3 tier** will **RUN** .

How They Communicate ?

They will communicate **internally** using called as **Service** .

For Example if I'm user I want **routing** I use

- / it will go to **front end** ,
- /api then it will go to **backend** for that I use **ingress controller** .
- Load balancing we need **ALB** (Application load balancer)

First we set up Workspace (using EC2)

- **Launch** instance .
- **Key pair** also create it .

- **Copy SSH client example .**

Open terminal : SSH it

```
shubham@Shubhams-MBP Downloads % sudo ssh -i "3-tier-hq-key.pem" ubuntu@ec2-18-219-144-49.us-east-2.compute.amazonaws.com
Password:
The authenticity of host 'ec2-18-219-144-49.us-east-2.compute.amazonaws.com (18.219.144.49)' can't be established.
ED25519 key fingerprint is SHA256:t2vSk1HuX5S9t9wLWYzvvgLuLzLmT2LgvrFQ0yvejz8.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-18-219-144-49.us-east-2.compute.amazonaws.com' (ED25519) to the list of known hosts.
```

Now I want to **Clone Repository** :

```
ubuntu@ip-172-31-42-35:~$ git clone https://github.com/LondheShubham153/TWSThreeTierAppChallenge.git
Cloning into 'TWSThreeTierAppChallenge'...
remote: Enumerating objects: 56, done.
remote: Counting objects: 100% (56/56) done
```

Open SSH Client

Locate private key file

File permission : **chmod 400** <Key name >

Connect to instance : using Public DNS

```
ubuntu@ip-172-31-41-67:~(TWSThreeTierAppChallenge) [ssh]$ ls
TWSThreeTierAppChallenge
ubuntu@ip-172-31-42-35:~$ cd TWSThreeTierAppChallenge/
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge$ ls
README.md  backend  frontend  k8s_manifests
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge$
```

Now I want to **Create Docker file** :

Go to frontend file folder.

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ ls
package-lock.json  package.json  public  src
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$
```

Now you have to **asked developer** that **which version Nodejs** or **java script version** it will create.

React is a library

If you don't have information about project In **README** than go to **Package.json**

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ ls
package-lock.json  package.json  public  src
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ cat package.json
```

Here you get **little information** . about project.

Here we can see if I want to **start the project** the **command** is **Start jo** ki **Node** ki command hai .

```
{,
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build"
  }
}
```

- **Npm start** npm means (node package manger)

Or You can google it **react js app start command** .

- **Java script** code hai **NPM** use hoga
- **Node js** **Node** use hoga

DOCKERFILE

FRONTEND :

Now I Create Docker File :

```
ubuntu@ip-172-31-41-67:~/TWSThreeTierAppChallenge (zsh)
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ vim Dockerfile
```

You need **base image**

Node : 14 Stable base image version hai or you can ask developer .

App name ki **working directory** bna di

Packages ek **file** mai likhe rhte hai aur us file ko **copy** kr k **install** krna pdta hai .

For example: Nodejs ki **package.json** name ki file mai rhti hai .

Mai **copy** kruga **package .json** ko kha per **mere app k ander** . package* **name all file**

```
COPY package*.json ./
```

Package.json VS **package.log.json** hoti hai ? difference

Package.log.json mai **libraries k version lock** ho jate hai .

Let say react js ka version 1.7 chahiye **package.log.json** mai version 1.7 he milega

so it is better to copy both files that's y we use **package***

Now we take **copy our code** .

Npm install : packages install

Copy .. means → Copy every thing to your **current container** means **/app** because it is our working directory

```
COPY . .
```

Q. What is difference between **RUN** VS **CMD** ? **important interview Question**.

- **RUN** = basically **packages install** krna hai ya **library install** krna hai .
- **CMD** = **Container bnne k bad koi command run** krna hai toh mai **CMD** k ander dal duga

```
CMD ["npm", "start"]
```

This is My docker file → Forntend for

```
FROM node:14
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["npm", "start"]
```

Now we test it Build it :

@ Npm install used for node js libraries will install here

Very Important Question comes in mind why I use 2 times COPY command because every line in docker is a layer if I take first layer all code size will be big you can also do once copy is done than package .json you can remove .

Again one more Question `/app` kyu nhi likha because **working directory already we set** It so you don't need to write every time .

Why I take to **COPY command** because Every line in **Docker is a layer** . if I put all of them in **one line** than layer will become **Bigger. Size** Fast build hoga .

- `/app` Q nhi likha because **present working directory is / app** hai isliye to bar bar likhna nhi pdega .

Extra : You can use **multi stage docker** file also.

First we update system :

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ sudo apt-get update
```

Now we install Docker :

- **Sudo apt install docker.io** → Your docker will be installed.
- We get **Permission ERROR** because user **don't have permission to run docker command** .

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix
un/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/json":
ix /var/run/docker.sock: connect: permission denied
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ whoami
ubuntu
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$
```

User ko `/var/run/docker.sock` ka **owner bna dete** hai . toh user **docker ki command Run** kr skta hai .

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ sudo chown $USER /var/run/
docker.sock
```

Another WAY: if you add user to **docker Group** you have to **restart server**

- also you can add Docker group to our user . but you have to **restart the server** after that . that's y we use this way.

Q. How to **Check Current user** ? these are the Two ways .

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ whoami
ubuntu
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ echo $USER
ubuntu
```

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$
```

Build Image :

- Docker build -t <image tag> . → Dot means here my **Docker file** is **available** .
- docker build -t three-tier-frontend .

Three-tier-frontend our **image name** .

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ docker images
REPOSITORY      TAG       IMAGE ID       CREATED        SIZE
three-tier-frontend  latest   0374634b2941   14 seconds ago  1.12GB
node             14       1d12470fa662   9 months ago   912MB
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$
```

Now we **Run** these **image** by **default port** REACT-JS = 3000 port per chalta hai.

Our you can ask developer which port want to RUN .

Now we Create Container ?

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ docker run -d -p 3000:3000
three-tier-frontend:latest
6848f8cdd1cf4edcbf9e0493a5b6ab52d4fe81cb668d320a614ad852c8107edd
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$
```

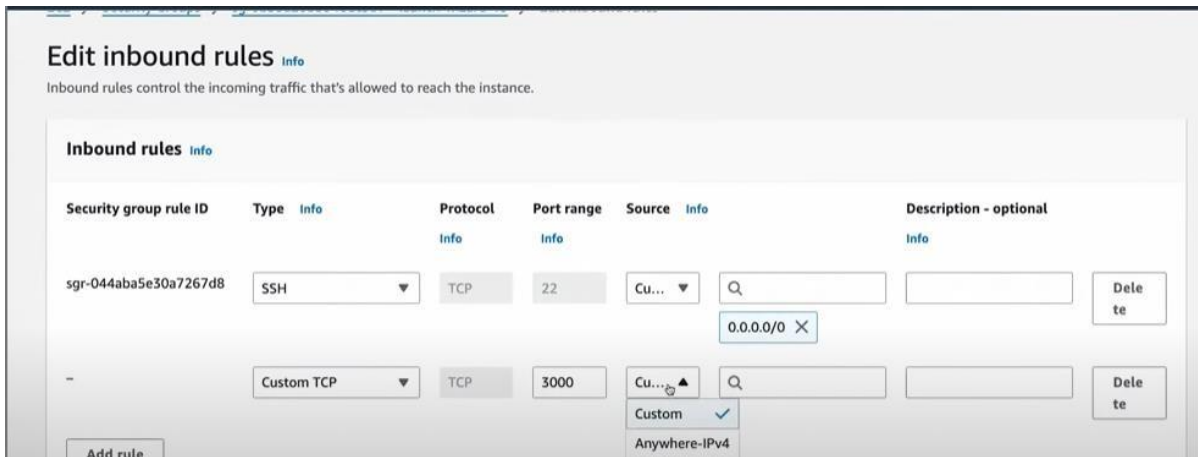
Port = host:container

Here we can see our **container RUNing**

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
6848f8cdd1cf   three-tier-frontend:latest   "docker-entrypoint.s..."   6 seconds ago   Up 4 seconds   0.0.0.0:3000->3000/tcp, :::3000->3000/tcp   funny_almeida
```

Now I Go to AWS instance

Security group mai ja k **port edit** kr skte hai vha **3000** save .



Go to Browser :

Google : <Public IP:3000>

so my **application** is **Running** . that means **Docker build** is **correct** .

Now I kill that container. Kam ho gya

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ docker kill 6848f8cdd1cf
6848f8cdd1cf
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$
```

Now I want to **Send these image to ECR** (Elastic container Registry)

For that **aws cli v2** install **hona** chaiye .

Always remember **Go to home directory** than **install** any thing .

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/frontend$ cd
ubuntu@ip-172-31-42-35:~$
```

Step 3: Install AWS CLI v2

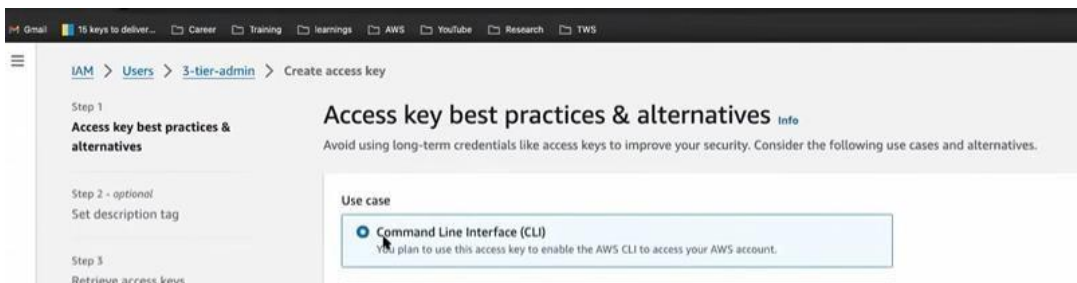
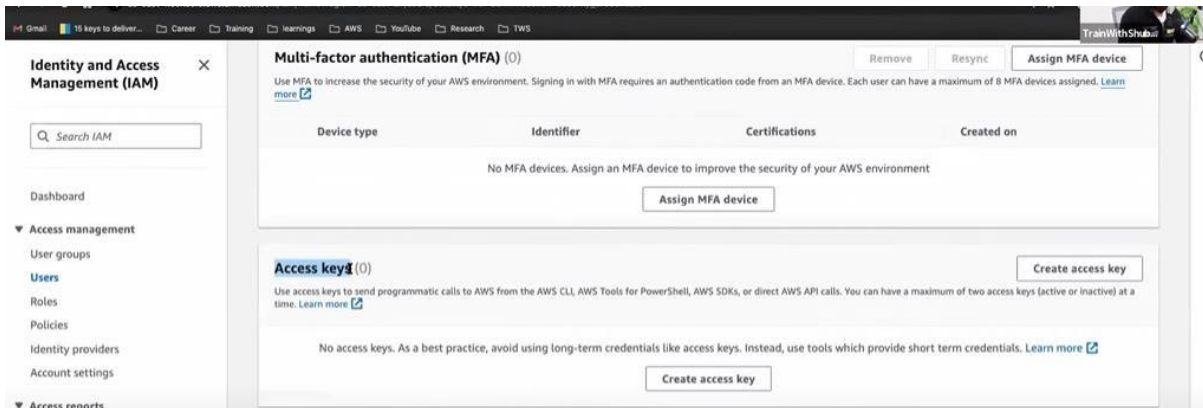
```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
sudo apt install unzip
unzip awscliv2.zip
sudo ./aws/install -i /usr/local/aws-cli -b /usr/local/bin --update
aws configure
```

Now you have to Configure **AWS CLI** by using **IAM** user **eks-admin** is **user name** . with **attach policy** (**Administrator access.**)

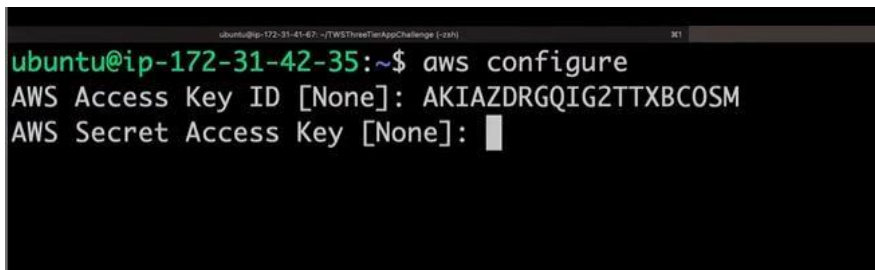
Step 1: IAM Configuration

- Create a user `eks-admin` with `AdministratorAccess`.
- Generate Security Credentials: Access Key and Secret Access Key.

Now I create Credentials : Security credentials go into (Go to IAM create access key)



Create it . Go to your server :

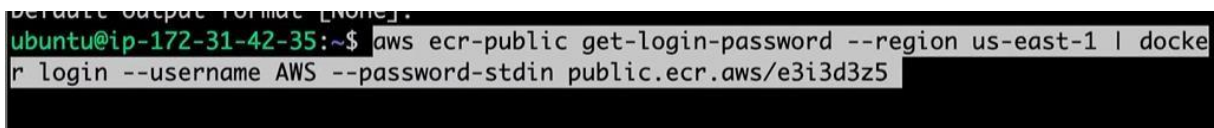


Go ECR :

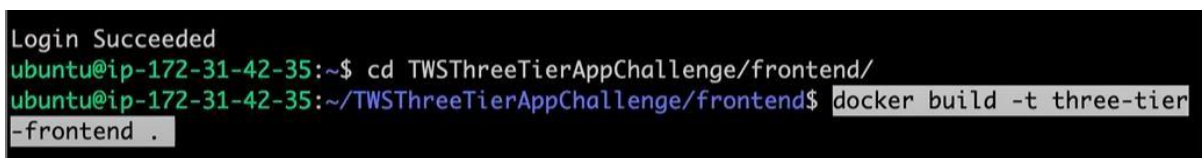
Create repository - 3 tier front end , public + Create.

How we push images to ECR for that you can see View Push commands

1. Command help you to login ECR using terminal .



Always remember whenever you build docker in that directory where you have dockerfile.



Now you want to **push image** for that you have to **TAG** it than **PUSH** it.

PUSH COMMAND :

1. Retrieve an authentication token and authenticate your Docker client to your registry.

Use the AWS CLI:

```
aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/e3i3d3z5
```

Note: If you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.

2. Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions [here](#). You can skip this step if your image is already built:

```
docker build -t three-tier-frontend .
```

3. After the build completes, tag your image so you can push the image to this repository:

```
docker tag three-tier-frontend:latest public.ecr.aws/e3i3d3z5/three-tier-frontend:latest
```

4. Run the following command to push this image to your newly created AWS repository:

```
docker push public.ecr.aws/e3i3d3z5/three-tier-frontend:latest
```

Now we go to BACKEND :

BACKEND :

```
ubuntu@ip-172-31-41-67: ~/TWSThreeTierAppChallenge [zsh]
ubuntu@ip-172-31-42-35: ~/TWSThreeTierAppChallenge/frontend$ cd ..
ubuntu@ip-172-31-42-35: ~/TWSThreeTierAppChallenge$ cd backend/
ubuntu@ip-172-31-42-35: ~/TWSThreeTierAppChallenge/backend$ ls
Dockerfile db.js index.js models package-lock.json package.json routes
```

Here you can see **index.js** .app lisening port 8080. Here you can see all APIs , port no. 8080

It means **when I run my application** it will **run index.js**

how we run? = **node index.js** jis se application run hogi . this application is **node**

if we have python application we writte here main.py just like

Create DockerFile For Backend

```
ubuntu@ip-172-31-42-35: ~/TWSThreeTierAppChallenge/backend$ vim Dockerfile
```

```
FROM node:14

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

CMD ["node","index.js"]
```

What is the **command to run application** ?

CMD =["node",< File name jo apki **entry point** hai >]

Go to ECR **Create new repository** = three tier **backend**

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/backend$ aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/e3i3d3z5
```

Push commands for three-tier-backend

Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see [Getting Started with Amazon ECR](#).

Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see [Registry Authentication](#).

1. Retrieve an authentication token and authenticate your Docker client to your registry.

Use the AWS CLI:

```
aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/e3i3d3z5
```

Note: If you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.

2. Build your Docker image using the following command. For information on building a Docker file from scratch see instructions [here](#). You can skip this step if your image is already built:

```
docker build -t three-tier-backend .
```

3. After the build completes, tag your image so you can push the image to this repository:

```
docker tag three-tier-backend:latest public.ecr.aws/e3i3d3z5/three-tier-backend:latest
```

4. Run the following command to push this image to your newly created AWS repository:

```
docker push public.ecr.aws/e3i3d3z5/three-tier-backend:latest
```

After I **build image** Now I want to run it . so container will created .

```
Successfully tagged three-tier-backend:latest
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/backend$ docker run -d -p 8080:8080 three-tier-backend:latest
646b8817a63b95a154b1b7cd36bbe5c5f4ccd1c2c12da43f310f964c5f407dbf
```

Now I do **docker ps** I can see my container .

If I want to **check backend run fine or not** than I do **docker logs <container id>**

ERROR we can see here .

because our **backend** is good but it is **not connected** with **Mongodb database** .

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/backend$ docker logs 646b88
Listening on port 8080...
Could not connect to database. MongooseError: The `uri` parameter to `openUri()` must be a string, got "undefined". Make sure the first parameter to `mongoose.connect()` or `mongoose.createConnection()` is a string.
```

For Now I just **push this image to ECR**.

When we do **Kubernetes deployment** at that time **mongo db** k sath **integrate kr dege** .

Same **TAG** . than **PUSH** it .

Now what we do **MONGO-DB** ko directly **Kubernetes** k sath chalayega .

Create Cluster K8's :

Now we **create k8's Cluster** . = **EKS**

For that if you want to **create EKS cluster** mai bnane k liye **EKSctl** name **tool** chahiye .

Note : **/usr/local/bin** -> help to run command any where if you facing issue just move script file to that location Executable file

Step 5: Install kubectl

```
curl -o kubectl https://amazon-eks.s3.us-west-2.amazonaws.com/1.19.6/2021-01-05/bin/linux/amd64
chmod +x ./kubectl
sudo mv ./kubectl /usr/local/bin
kubectl version --short --client
```

Step 6: Install eksctl

```
curl --silent --location "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl"
sudo mv /tmp/eksctl /usr/local/bin
eksctl version
```

Kubectl k8's cluster ko control krta hai . **always remember go to home directory .than install it .**

```
ubuntu@ip-172-31-41-67:~/TWSThreeTierAppChallenge (csh)
ubuntu@ip-172-31-42-35:~$ curl -o kubectl https://amazon-eks.s3.us-west-2.amazonaws.com/1.19.6/2021-01-05/bin/linux/amd64/kubectl
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 57.4M  100 57.4M    0     0  19.8M      0  0:00:02  0:00:02 --:--:-- 19.8M
```

Why we move/bin to /usr/local/bin directoty **shell script** .

Because I don't want to every time give ./ (dot slash)

--silent flag means it will **not show any logs** in terminal when we download any thing it will show some logs in terminal .

Extra info .

- EKS se cluster bnate ho .
- kubectl se cluster control krte ho.

#

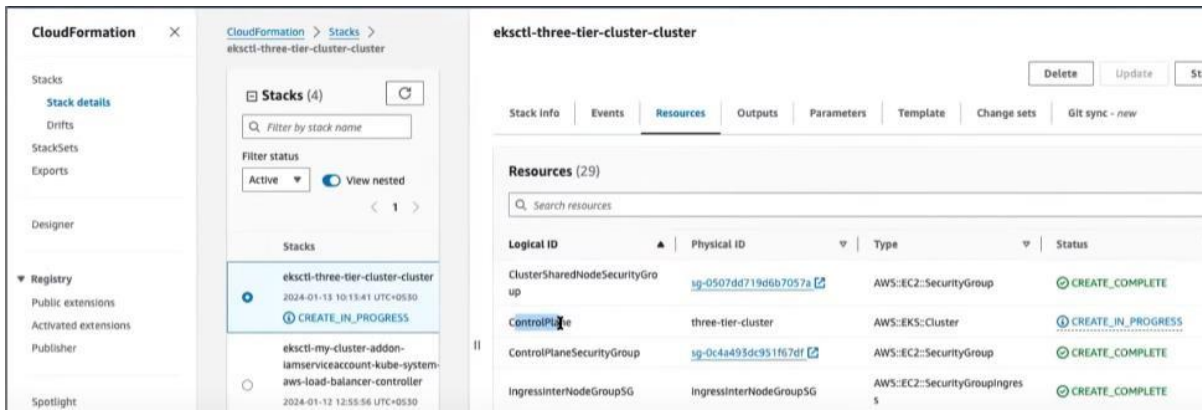
SET UP EKS Cluster

- eksctl create cluster --name three-tier-cluster --region us-west-2 --node-type t2.medium --nodes-min 2 --nodes-max 2
- kubectl get nodes

Extra Tip:

You can see **Stack** jo bn rha hai o **Cloud formation** se bn rha hai control plane .

#



Let say **particular cluster k nodes** chahiye muzhe

Command use :

- **kubectl get nodes**

@ Let say I want **particular cluster k nodes** chahiye .means kubectl ko **bind krna hai eks cluster k sath**

- **aws eks update-kubeconfig --region us-west-2 --name **three-tier-cluster****

Now we want to **Create Kubernetes Manifest** :

Easy way to **Create Manifest** go to google search it Kubernetes deployment .

If you want to **run nginx inside k8's** than **spec = image nginx**

backend-deployment.yaml

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: api
5    namespace: workshop
6    labels:
7      role: api
8      env: demo
9  spec:
10   replicas: 1
11   strategy:
12     type: RollingUpdate
13     rollingUpdate:
14       maxSurge: 1
15       maxUnavailable: 25%

```

```

16   selector:
17     matchLabels:
18       role: api
19   template:
20     metadata:
21       labels:
22         role: api

```

```

23     spec:
24       containers:
25       - name: api
26         image: public.ecr.aws/w8u5e4v2/workshop-backend:v1
27         imagePullPolicy: Always
28         env:
29         - name: MONGO_CONN_STR
30           value: mongodb://mongodb-svc:27017/todo?directConnection=true
31         - name: MONGO_USERNAME
32           valueFrom:
33             secretKeyRef:
34               name: mongo-sec
35               key: username

```

todo

This is the **database name**.

Instead of connecting to the default database, the application connects to todo : If the todo database doesn't exist yet, MongoDB can create it automatically when data is inserted.

?directConnection=true

This is a **connection option**.

It tells the MongoDB client:

Connect directly to this MongoDB server instead of trying to discover a replica set or cluster.

In many applications, you'll see:

mongodb://mongodb-svc:27017/todo?directConnection=true

Here:

- mongodb:// → Protocol
- mongodb-svc → Service name (hostname)
- 27017 → MongoDB port
- **todo** → Database name
- directConnection=true → Connect directly to this MongoDB server

```

36         - name: MONGO_PASSWORD
37           valueFrom:
38             secretKeyRef:
39               name: mongo-sec
40               key: password
41       ports:
42       - containerPort: 8080
43       livenessProbe:
44         httpGet:
45           path: /ok
46           port: 8080
47       initialDelaySeconds: 2

```



```

48         periodSeconds: 5
49         readinessProbe:
50             httpGet:
51                 path: /ok
52                 port: 8080
53             initialDelaySeconds: 5
54             periodSeconds: 5
55             successThreshold: 1

```

Now Lets understand MONGODB .

Deploy.yaml

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4      namespace: workshop
5      name: mongodb
6  spec:
7      replicas: 1
8      selector:
9          matchLabels:
10             app: mongodb
11      template:
12          metadata:
13              labels:
14                  app: mongodb
15          spec:
16              containers:
17                  - name: mongodb
18                    image: mongo:4.4.6

```

```

19         command:
20             - "numactl"
21             - "--interleave=all"
22             - "mongod"
23             - "--wiredTigerCacheSizeGB"
24             - "0.1"
25             - "--bind_ip"
26             - "0.0.0.0"

```

```

27         ports:
28             - containerPort: 27017
29         resources:
30             requests:
31                 memory: "512Mi"
32                 cpu: "250m"
33             limits:
34                 memory: "1Gi"
35                 cpu: "500m"
36         env:
37             - name: MONGO_INITDB_ROOT_USERNAME
38               valueFrom:
39                 secretKeyRef:
40                     name: mongo-sec
41                     key: username
42             - name: MONGO_INITDB_ROOT_PASSWORD
43               valueFrom:

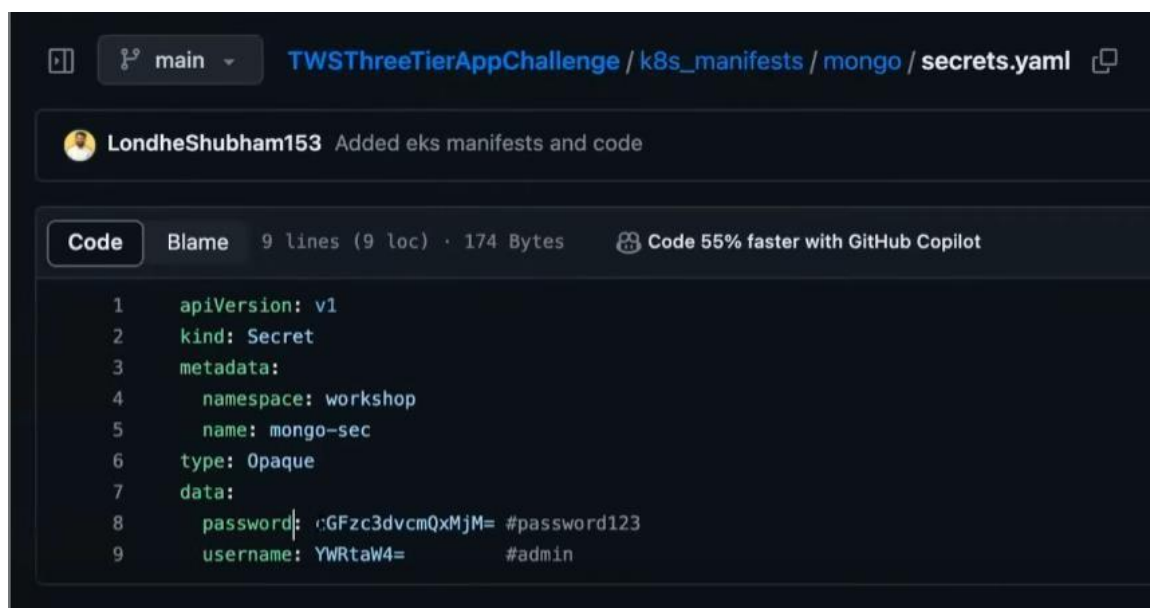
```

```

35         - name: MONGO_INITDB_ROOT_PASSWORD
36           valueFrom:
37             secretKeyRef:
38               name: mongo-sec
39               key: password
40       volumeMounts:
41         - name: mongo-volume
42           mountPath: /data/db
43     volumes:
44     - name: mongo-volume
45       persistentVolumeClaim:
46         claimName: mongo-volume-claim

```

Mongo db k ander **secret** aa rhe hai toh hme [secrets.yaml](#) bnane pdegi



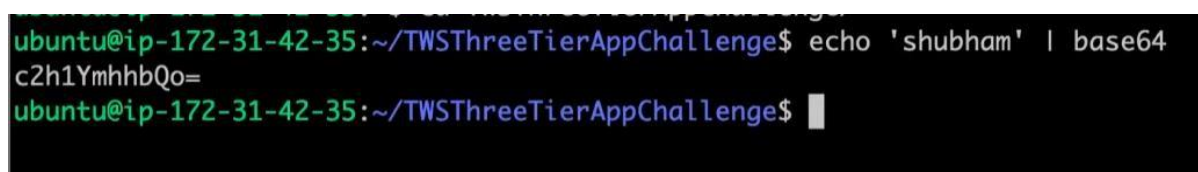
```

1  apiVersion: v1
2  kind: Secret
3  metadata:
4    namespace: workshop
5    name: mongo-sec
6  type: Opaque
7  data:
8    password: cGFzc3dvcmQxMjM= #password123
9    username: YWRtaW4= #admin

```

Secrete file bnani hai khud ka password dalna hai .

Q. How many knows this **Password how we create it** ? for example .

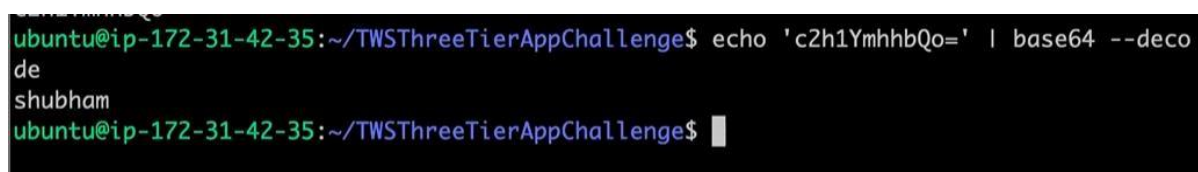


```

ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge$ echo 'shubham' | base64
c2h1YmhhbQo=
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge$

```

If I want to **decode** it .



```

ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge$ echo 'c2h1YmhhbQo=' | base64 --deco
de
shubham
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge$

```

Now we do **MONGO DB Deployment**. For that you have to go [MONGO DB folder](#)

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ ls
backend-deployment.yaml  frontend-deployment.yaml  full_stack_lb.yaml
backend-service.yaml     frontend-service.yaml     mongo
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ cd mongo/
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ ls
deploy.yaml  secrets.yaml  service.yaml
```

Here you see **3 files**

Lets **check Service file - mongodb**

cat service.yaml

```
apiVersion: v1
kind: Service
metadata:
  namespace: workshop
  name: mongodb-svc
spec:
  selector:
    app: mongodb
  ports:
  - name: mongodb-svc
    protocol: TCP
    port: 27017
    targetPort: 27017ubuntu@ip-172-31-42-35:~/T
```

Service basically apke **mongo db** ko baki ki application/ Deployment **access kr paye** is liye hoti hai 27017 jis mai mongo db chl rh hai uska **access** ek particular name ko **mongodb-svc**

If I want **backend k through database ko connect** krna hai toh meri **URL hogi mongo db ki mongodb//mongodb-svc**

Y sab internal access k liye hota hai .

Service make sure **internal application** ek dusre se **baat kr paye** .

@Generally **internal Communication** k liye bna hai **service** external k liye **nhi** .

External k liye **Ingress** hota hai . **load balancer**.

- **Deployment** ka kam **container ko RUN** krna
 - **Secrets** = login password / user
 - **Service** = **internal** communication , deployment **acces krna baki ki application** ko.
-

Now lets understand how we **run all these 3 file** .

- Kubectl apply -f **deploy.yaml** (**mongodb deployment**) ----- inside this we **already mention namespace**
- Kubectl **create** namespace **workspace**.

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl create namespace workshop
namespace/workshop created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl apply -f deploy.yaml
deployment.apps/mongodb created
```

But you can see **mongo db** ready nhi ho rha because **secrete pass** nhi hue abhi tak .

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl get deployment -n workshop
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
mongodb	0/1	1	0	35s

For that **Run** **secrete.yaml** file

- **Kubectl apply -f secrets.yaml**

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl get pods -n workshop
```

NAME	READY	STATUS	RESTARTS	AGE
mongodb-797968fbc5-m2hfw	1/1	Running	0	81s

Now my **MongoDB** is now RUNNING . and Now run **service.yaml**

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl apply -f service.yaml
service/mongodb-svc created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$
```

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl get svc -n workshop
```

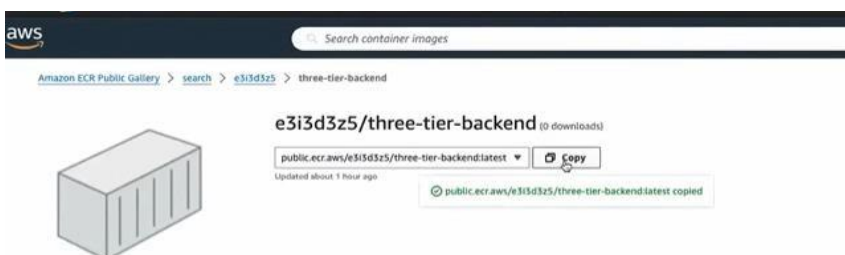
NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
mongodb-svc	ClusterIP	10.100.214.208	<none>	27017/TCP	21s

Service ka **type** nhi likhte ho by default Cluster ip leta hai .

Now we run backend :

Backend service :

Just update the image name



```

rollingUpdate:
  maxSurge: 1
  maxUnavailable: 25%
selector:
  matchLabels:
    role: api
template:
  metadata:
    labels:
      role: api
  spec:
    containers:
      - name: api
        image: public.ecr.aws/e3i3d3z5/three-tier-backend:latest
        imagePullPolicy: Always
        env:
          - name: MONGO_CONN_STR
            value: mongodb://mongodb-svc:27017/todo?directConnection=true
          - name: MONGO_USERNAME
            valueFrom:

```

```

      - name: MONGO_PASSWORD
        valueFrom:
          secretKeyRef:
            name: mongo-sec
            key: password
    ports:
      - containerPort: 8080
    livenessProbe:
      httpGet:
        path: /ok
        port: 8080
      initialDelaySeconds: 2
      periodSeconds: 5
    readinessProbe:
      httpGet:
        path: /ok
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 5
      successThreshold: 1

```

Lets check **backend service file:**

```

--
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: workshop
spec:
  ports:
    - port: 8080
      protocol: TCP
  type: ClusterIP
  selector:
    role: api
~
~
~

```


Mongo db k service ka naam ky tha ? → **api** tha

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ kubectl apply -f backend-deployment.yaml
deployment.apps/api created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ kubectl apply -f backend-service.yaml
service/api created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$
```

- Kubectl get pods -n workshop

```
workshop
NAME                                READY   STATUS    RESTARTS   AGE
api-64fb8488f8-xq4mk                1/1     Running   0           30s
mongodb-797968fbc5-m2hfw            1/1     Running   0           7m15s
```

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ kubectl logs api-64fb8488f8-xq4mk -n workshop
Listening on port 8080...
Connected to database.
```

NOW our **database** tier and **logical** tier bn gya hai .

one tier is remaining which is **web tier**.

- **Bahr se kisi ko access** dena hai website ka toh **load balancer ALB** lgana pdega .
- **Ingress controller** internal routing k liye .

Deployment feature **autoscaling**

Now **Frontend deployment** :

Vi frontend-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: workshop
  labels:
    role: frontend
    env: demo
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 25%
  selector:
    matchLabels:
      role: frontend
  template:
    metadata:
```

How to take image from ECR ?

- Go to **ECR + front end** repository + **view public listing** + Copy .

```
template:
  metadata:
    labels:
      role: frontend
  spec:
    containers:
      - name: frontend
        image: public.ecr.aws/e3i3d3z5/three-tier-frontend:latest
        imagePullPolicy: Always
        env:
          - name: REACT_APP_BACKEND_URL
            value: "http://challenge.trainwithshubham.com/api/tasks"
        ports:
          - containerPort: 3000
```

Now lets check **frontend service** :

```
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: workshop
spec:
  ports:
    - port: 3000
      protocol: TCP
  type: ClusterIP
  selector:
    role: frontend
```

Backend -- **3000 port** ---database

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ kubectl apply -f frontend-deployment.yaml
deployment.apps/frontend created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$
```

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ kubectl apply -f frontend-service.yaml
service/frontend created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$
```

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ kubectl get pods -n workshop
NAME                                READY   STATUS    RESTARTS   AGE
api-64fb8488f8-xq4mk                1/1     Running   0           12m
frontend-6c48985-bqzjl              1/1     Running   0           36s
mongodb-797968fbc5-m2hfw            1/1     Running   0           19m
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$
```

Here you can see logical tier+ DB tier + front tier all of them are running .

How to Access these Application .

For that you need **ingress controller** jo y **Routing** kr k dega .

Install Ingress using Helm .

Helm k through app **ingress controller** install kroge .

What is helm ? is kind of **package manager** , You have to install it **helm package manager** .

Load balancer ki **IAM policy** rhti hai usko **download** kruga .

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ cd
ubuntu@ip-172-31-42-35:~$ curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.5.4/docs/install/iam_policy.json
```

Now I am going to create **IAM policy** jisse **EKS** and **load balancer** ko connectivity mil jaye.

[Resources,allow : target group]

Step 9: Install AWS Load Balancer

- curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.5.4/docs/install/iam_policy.json
- aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --policyForEKS-document file://iam_policy.json
- eksctl utils associate-iam-oidc-provider --region=us-west-2 --cluster=three-tier-cluster --approve
- eksctl create iamserviceaccount --cluster=three-tier-cluster --namespace=kube-system --name=aws-load-balancer-controller --role-name AmazonEKSLoadBalancerControllerRoleForEKS --attach-policy-arn=arn:aws:iam::626072240565:policy/AWSLoadBalancerControllerIAMPolicyForEKS --approve --region=us-west-2

service account : one service communicate with other service

remember you can change **policy name** # **626072240565** aws account number

Now , My load balancer service account is created

Step 9: Install AWS Load Balancer

```
curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.5.4/
aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --policy-document file:
eksctl utils associate-iam-oidc-provider --region=us-west-2 --cluster=three-tier-cluster --app
eksctl create iamserviceaccount --cluster=three-tier-cluster --namespace=kube-system --name=aw
```

Now that **load balancer** ko **K8's k ander deploy** krna hai .

I install Helm then using helm I install EKS chart

Step 10: Deploy AWS Load Balancer Controller

```
sudo snap install helm --classic
helm repo add eks https://aws.github.io/eks-charts
helm repo update eks
helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system --se
kubectl get deployment -n kube-system aws-load-balancer-controller
kubectl apply -f full_stack_lb.yaml
```

```
ubuntu@ip-172-31-42-35:~$ sudo snap install helm --classic
helm 3.10.1 from Snapcrafters installed
ubuntu@ip-172-31-42-35:~$ helm repo add eks https://aws.github.io/eks-charts
```

```
ubuntu@ip-172-31-42-35:~$ helm repo list
NAME      URL
eks       https://aws.github.io/eks-charts
ubuntu@ip-172-31-42-35:~$
```

Lets **check** load balancer running or not .

Aws load balancer controller file helm create and install it for me I do not need to create it

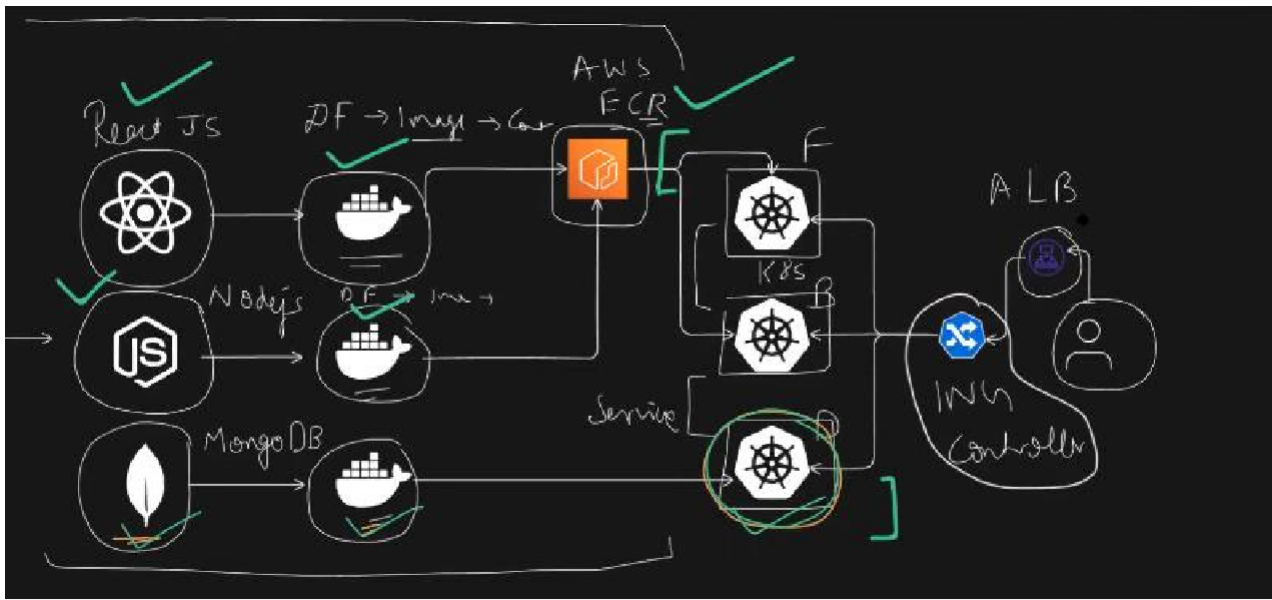
If I want to create **ingress** than I use **Workshop** namespace

But I want to create for whole **cluster** inside than I use **kubsystem**

Load balancer is **service** AWS we **associate** with **Cluster**.

Check load balancer is running or Not ? replica count 2 that y it show 2

```
ubuntu@ip-172-31-42-35:~$ kubectl get deployment -n kube-system aws-load-bala
roller
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
aws-load-balancer-controller        2/2      2              2            47s
ubuntu@ip-172-31-42-35:~$
```

So basically what happen user use ALB but for Routing we use Ingress

Now we Create Ingress for Routing :

Lets create it

```
ubuntu@ip-172-31-42-35:~$ cd TWSThreeTierAppChallenge/
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge$ cd k8s_manifests/
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ vim full_stack_lb.yaml
```

```
--
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mainlb
  namespace: workshop
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}]'
spec:
  ingressClassName: alb
  rules:
    - host: app.trainwithshubham.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
```

Ingress k liye **networking.k8s.io** likhte hai

Mainlb **ingress** ka **name** hai .

```
ingressClassName: alb
rules:
  - host: challenge.trainwithshubham.com
    http:
      paths:
        - path: /api
          pathType: Prefix
          backend:
            service:
              name: api
              port:
                number: 8080
        - path: /
          pathType: Prefix
          backend:
```



```
service:
  name: frontend
  port:
    number: 3000
```

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ kubectl apply -f full_stack_lb.yaml
ingress.networking.k8s.io/mainlb created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$
```

```
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ kubectl get ing -n workshop
NAME      CLASS  HOSTS                                PORTS  AGE
mainlb    alb    challenge.trainwithshubham.com      80     37s
10311.us-west-2.elb.amazonaws.com      80     37s
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$
```

Godaddy :

- A = ip
- CNAME used for Custom domain

Add new record

< [Domain Portfolio](#)

trainwithshubham.com

[Overview](#) **[DNS](#)** [Products](#)

DNS Records [Forwarding](#) [Nameservers](#) [Premium DNS](#) [Hostnames](#) [DNSSEC](#)

DNS records define how your domain behaves, like showing your website content and delivering your email.

New Records

[CNAME records](#) are a type of subdomain, or alias, that points to another domain name.

Type *	Name *	Value *	TTL
CNAME	challenge	k8s-workshop-mainlb-7f430e3450-1036210	Custom
			Seconds
			600

[Add More Records](#) [Save](#) [Cancel](#)

Google it = enter domain name <http://challenge.trainwithshubham.com> you can see your website .

Complete !